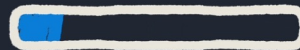


Module : Gradient Boosting

Instructor: Dr. Darshan
Ingle



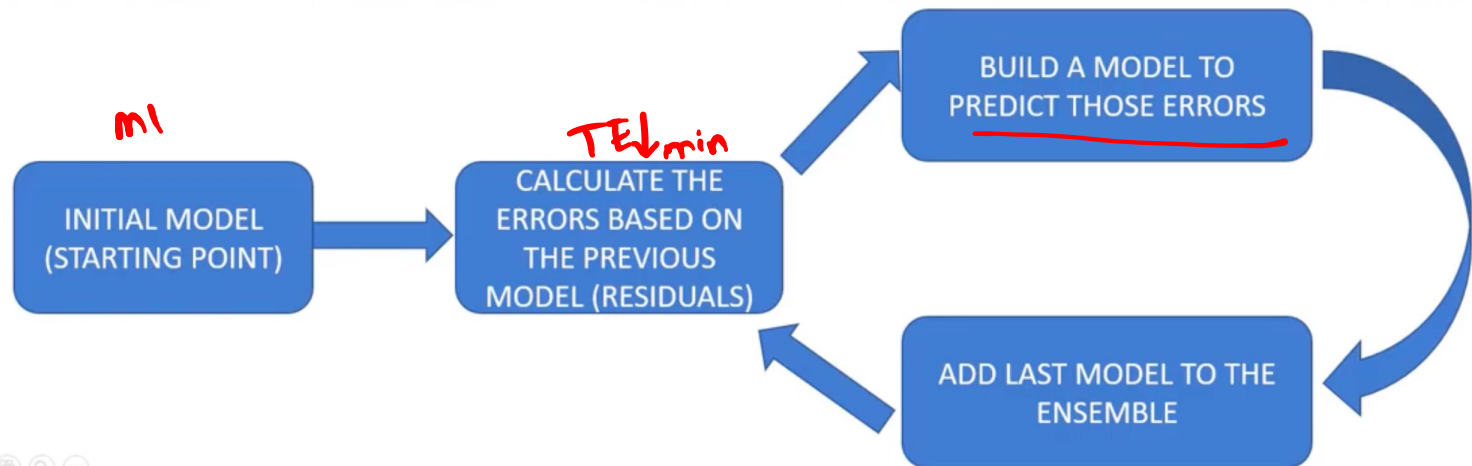
HI Y'ALL !!!



STARTING SOON

XGBOOST: STEPS

- XGBoost repeatedly builds new models and combine them into an ensemble model
- Initially build the first model and calculate the error for each observation in the dataset
- Then you build a new model to predict those residuals (errors)
- Then you add prediction from this model to the ensemble of models
- XGboost is superior compared to gradient boosting algorithm since it offers a good balance between bias and variance (Gradient boosting only optimized for the variance so tend to overfit training data while Xgboost offers regularization terms that can improve model generalization).



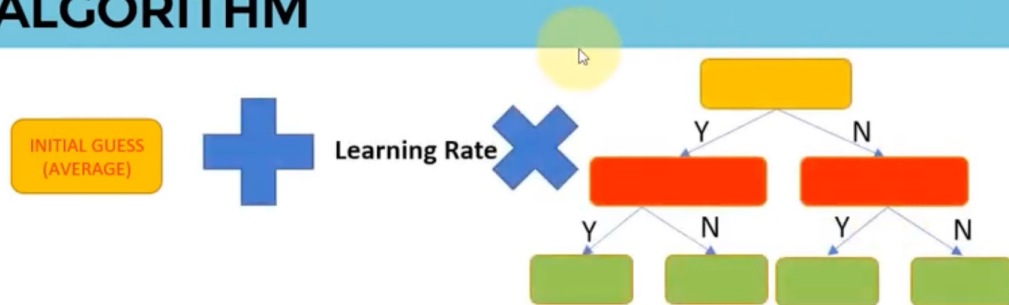
XGBOOST: GRADIENT BOOSTING ALGORITHM



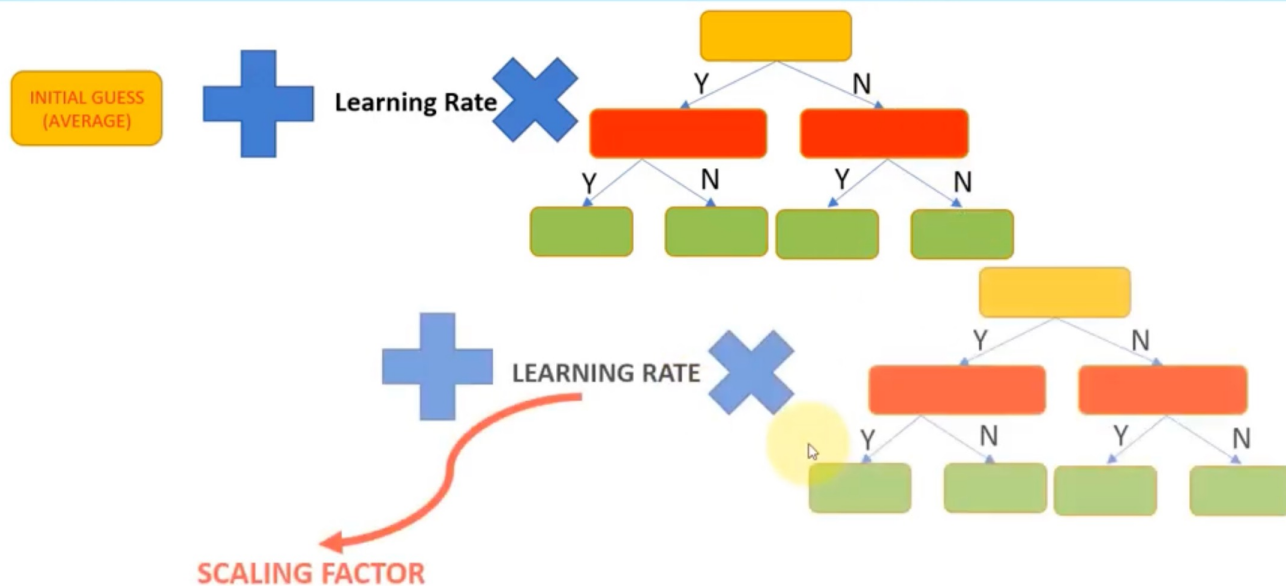
INITIAL GUESS
(AVERAGE)



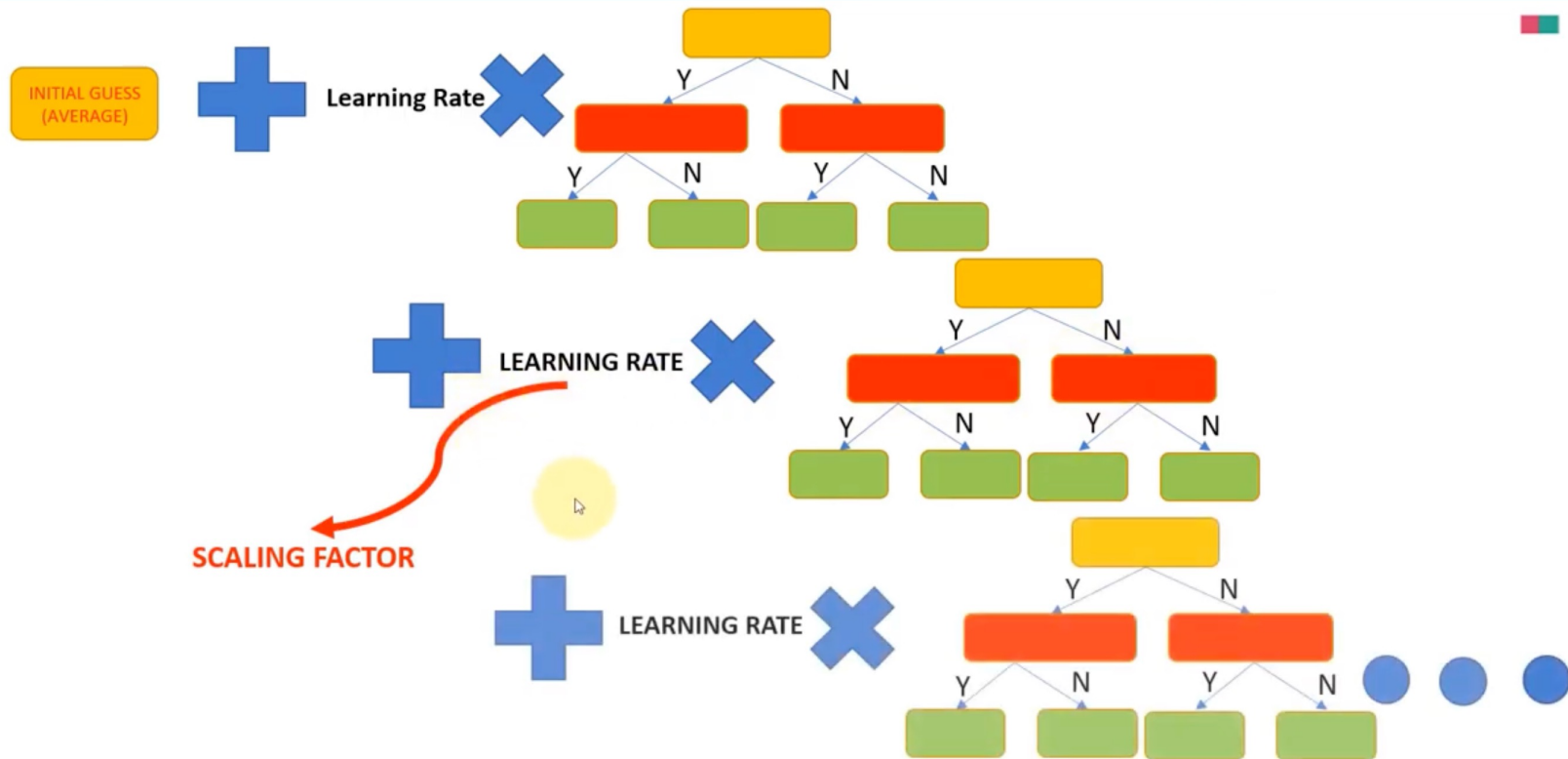
XGBOOST: GRADIENT BOOSTING ALGORITHM



XGBOOST: GRADIENT BOOSTING ALGORITHM



XGBOOST: GRADIENT BOOSTING ALGORITHM

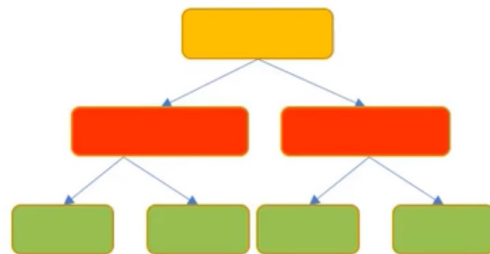


XGBOOST: GRADIENT BOOSTING ALGORITHM



- Gradient boost works by building a tree based on the error (residuals) from the previous tree. 
- Gradient boost scales the trees and then adds the predictions from the new tree to the predictions from previous trees.
- [Example adopted from the awesome StatQuest \(by Josh Starmer\):](https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s)
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

X_1	X_2	X_3	y
Height	Color	Gender	Weight (Kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

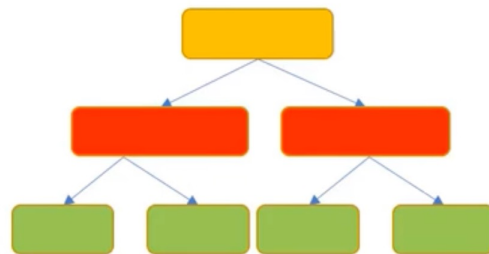


XGBOOST: GRADIENT BOOSTING ALGORITHM



- Gradient boost works by building a tree based on the error (residuals) from the previous tree. 
- Gradient boost scales the trees and then adds the predictions from the new tree to the predictions from previous trees.
- [Example adopted from the awesome StatQuest \(by Josh Starmer\):](https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s)
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

Height	Color	Gender	Weight (Kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57



INPUT FEATURES

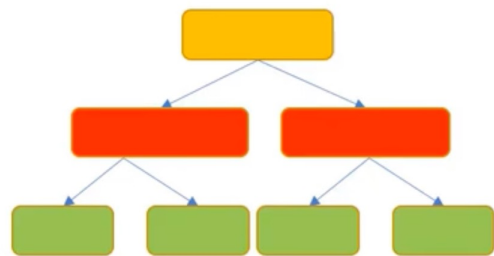
XGBOOST: GRADIENT BOOSTING ALGORITHM



- Gradient boost works by building a tree based on the error (residuals) from the previous tree. 
- Gradient boost scales the trees and then adds the predictions from the new tree to the predictions from previous trees.
- [Example adopted from the awesome StatQuest \(by Josh Starmer\):](https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s)
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

X₁ X₂ X₃ y p

Height	Color	Gender	Weight (Kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57



avg = 71.2

VARIABLE TO BE
PREDICTED

INPUT FEATURES

XGBOOST: GRADIENT BOOSTING ALGORITHM



- Let's assume that the initial model predictions (starting point) is the average weight is 71.2
- Gradient boost builds a tree based on the error from the first tree.
- The tree is built by assuming that the features (heights, color, and gender) predicts the residuals (new column that we have just created).



71.2

**INITIAL STARTING
POINT (PREDICTIONS)**

Example adopted from the awesome StatQuest (by Josh Starmer):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

XGBOOST: GRADIENT BOOSTING ALGORITHM



- Let's assume that the initial model predictions (starting point) is the average weight is 71.2
- Gradient boost builds a tree based on the error from the first tree.
- The tree is built by assuming that the features (heights, color, and gender) predicts the residuals (new column that we have just created).

71.2

**INITIAL STARTING
POINT (PREDICTIONS)**

Height	Color	Gender	Weight (Kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

INPUT FEATURES

Example adopted from the awesome StatQuest (by Josh Starmer):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

XGBOOST: GRADIENT BOOSTING ALGORITHM

- Let's assume that the initial model predictions (starting point) is the average weight is 71.2
- Gradient boost builds a tree based on the error from the first tree.
- The tree is built by assuming that the features (heights, color, and gender) predicts the residuals (new column that we have just created).

71.2

INITIAL STARTING
POINT (PREDICTIONS)

x_1 Height	x_2 Color	x_3 Gender	y Weight (Kg)	p Error = True – predicted
1.6	Blue	Male	88	$88 - 71.2 = 16.8$
1.6	Green	Female	76	$76 - 71.2 = 4.8$
1.5	Blue	Female	56	$56 - 71.2 = -15.2$
1.8	Red	Male	73	$73 - 71.2 = 1.8$
1.5	Green	Male	77	$77 - 71.2 = 5.8$
1.4	Blue	Female	57	$57 - 71.2 = -14.2$

INPUT FEATURES

ERRORS
(RESIDUALS)

Mean
71.2

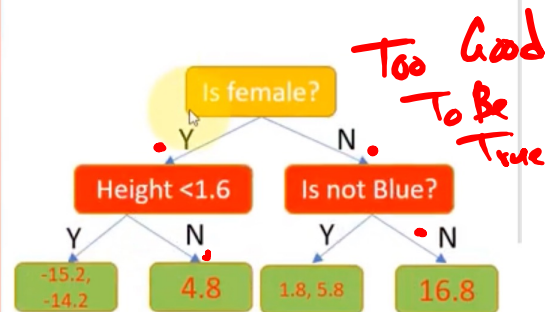
Example adopted from the awesome StatQuest (by Josh Starmer):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

XGBOOST: GRADIENT BOOSTING ALGORITHM

- Let's assume that the initial model predictions (starting point) is the average weight is 71.2
- Gradient boost builds a tree based on the error from the first tree.
- The tree is built by assuming that the features (heights, color, and gender) predicts the residuals (new column that we have just created).

71.2
**INITIAL STARTING
POINT (PREDICTIONS)**

Height	Color	Gender	Weight (Kg)	Error = True - predicted
1.6	Blue	Male	88	$88 - 71.2 = 16.8$
1.6	Green	Female	76	$76 - 71.2 = 4.8$
1.5	Blue	Female	56	$56 - 71.2 = -15.2$
1.8	Red	Male	73	$73 - 71.2 = 1.8$
1.5	Green	Male	77	$77 - 71.2 = 5.8$
1.4	Blue	Female	57	$57 - 71.2 = -14.2$



INPUT FEATURES

**ERRORS
(RESIDUALS)**

Example adopted from the awesome StatQuest (by Josh Starmer):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

XGBOOST: GRADIENT BOOSTING ALGORITHM



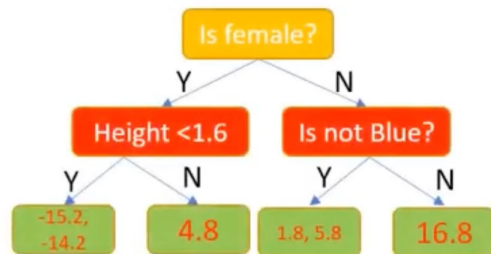
- Note that the number of leaves is restricted to 4 in this example for the sake of simplicity.
- Let's replace the values with the average a shown below.

Example adopted from the awesome StatQuest (by Josh Starmer):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

XGBOOST: GRADIENT BOOSTING ALGORITHM



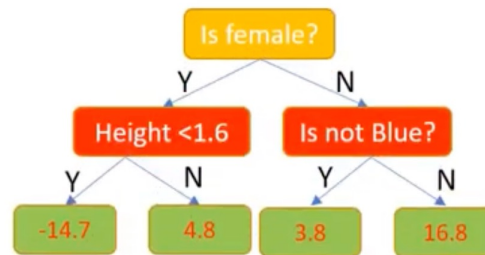
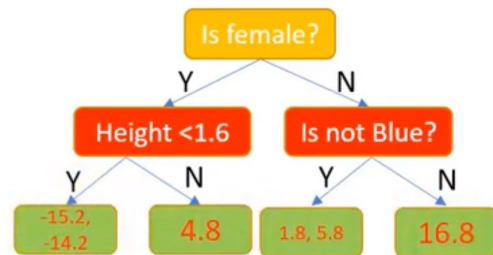
- Note that the number of leaves is restricted to 4 in this example for the sake of simplicity.
- Let's replace the values with the average a shown below.



Example adopted from the awesome StatQuest (by Josh Starmer):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

XGBOOST: GRADIENT BOOSTING ALGORITHM

- Note that the number of leaves is restricted to 4 in this example for the sake of simplicity.
- Let's replace the values with the average a shown below.



$$Average_1 = (-15.2 - 14.2)/2 = -14.7$$

$$Average_2 = (1.8 + 5.8)/2 = 3.8$$

Example adopted from the awesome StatQuest (by Josh Starmer):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

XGBOOST: GRADIENT BOOSTING ALGORITHM



- Now that we have built a tree, let's combine the previous predictions with the new tree to generate new predictions!

71.2

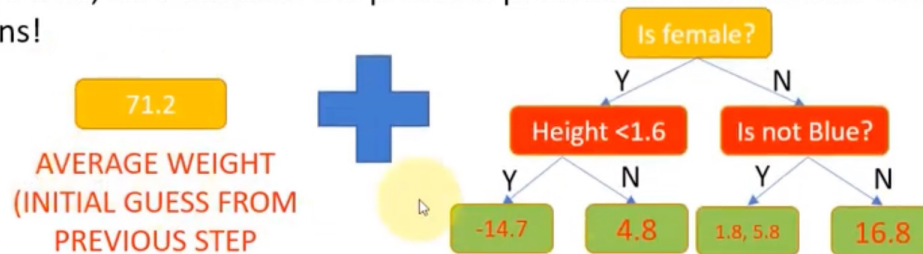
AVERAGE WEIGHT
(INITIAL GUESS FROM
PREVIOUS STEP)

Height	Color	Gender	Weight (Kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

Example adopted from the awesome StatQuest (by Josh Starmer):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

XGBOOST: GRADIENT BOOSTING ALGORITHM

- Now that we have built a tree, let's combine the previous predictions with the new tree to generate new predictions!



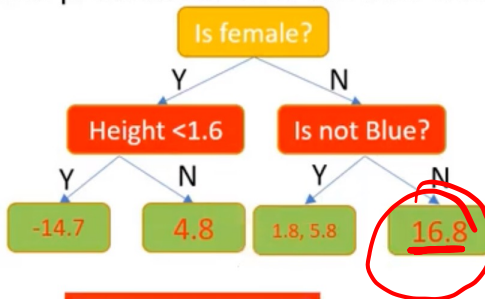
Height	Color	Gender	Weight (Kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

Example adopted from the awesome StatQuest (by Josh Starmer):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

XGBOOST: GRADIENT BOOSTING ALGORITHM

- Now that we have built a tree, let's combine the previous predictions with the new tree to generate new predictions!

71.2
AVERAGE WEIGHT
(INITIAL GUESS FROM
PREVIOUS STEP)



Height	Color	Gender	Weight (Kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

New predictions
88
⋮
⋮
⋮
⋮
⋮

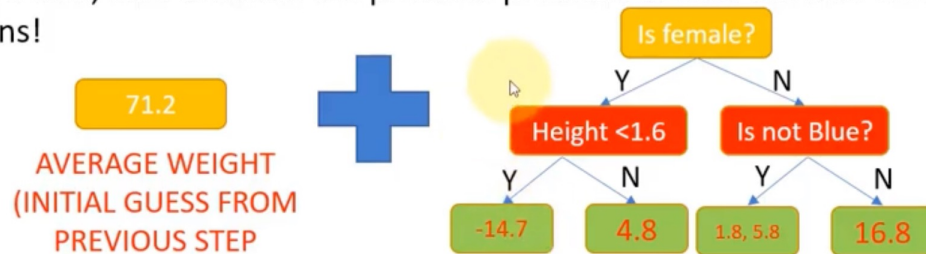
= 0

Predictions = 71.2 + 16.8 = 88

Example adopted from the awesome StatQuest (by Josh Starmer):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

XGBOOST: GRADIENT BOOSTING ALGORITHM

- Now that we have built a tree, let's combine the previous predictions with the new tree to generate new predictions!



Height	Color	Gender	Weight (Kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

New predictions
88
76
56
73
77
57

The model predictions match the true weight. This indicates that the model is overfitting the training data

$$\text{Predictions} = 71.2 + 16.8 = 88$$

71.2

Example adopted from the awesome StatQuest (by Josh Starmer):
<https://www.youtube.com/watch?v=3CC4N4z3GJc&t=87s>

XGBOOST: GRADIENT BOOSTING ALGORITHM

- We add a learning rate (range from 0 to 1) to overcome this issue.
- This parameter is used for scaling purposes by adjusting the newly added information from the new tree.
- Adding this tree and scaling it with the learning rate helps us get a little closer to the true values.
- By taking smaller steps, the model results in better predictions on the testing dataset (low variance).

71.2

AVERAGE WEIGHT
(INITIAL GUESS FROM
PREVIOUS STEP)

Height	Color	Gender	Weight (Kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

XGBOOST: GRADIENT BOOSTING ALGORITHM

$\eta = 0.1, 0.001, 0.01$

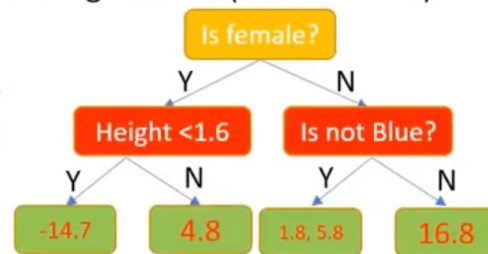
- We add a learning rate (range from 0 to 1) to overcome this issue.
- This parameter is used for scaling purposes by adjusting the newly added information from the new tree.
- Adding this tree and scaling it with the learning rate helps us get a little closer to the true values.
- By taking smaller steps, the model results in better predictions on the testing dataset (low variance).

71.2
AVERAGE WEIGHT
(INITIAL GUESS FROM
PREVIOUS STEP)



LEARNING RATE

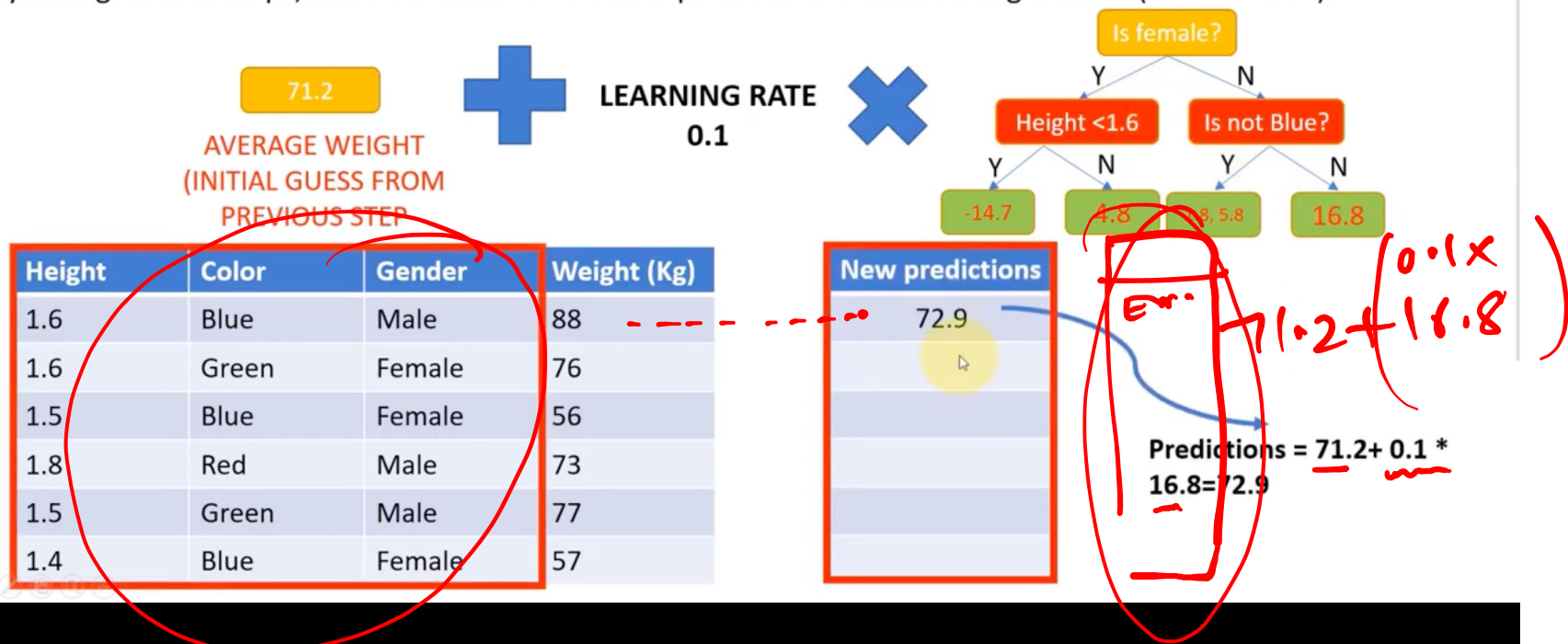
0.1



Height	Color	Gender	Weight (Kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

XGBOOST: GRADIENT BOOSTING ALGORITHM

- We add a learning rate (range from 0 to 1) to overcome this issue.
- This parameter is used for scaling purposes by adjusting the newly added information from the new tree.
- Adding this tree and scaling it with the learning rate helps us get a little closer to the true values.
- By taking smaller steps, the model results in better predictions on the testing dataset (low variance).



XGBOOST: GRADIENT BOOSTING ALGORITHM

- Now let's build another tree with the new residuals from the new predictions.

RECALL THAT THESE
ARE THE INITIAL
RESIDUALS

Predictions = $71.2 +$
 $0.1 * 16.8 = 72.9$

Height	Color	Gender	Weight (Kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57

New predictions	New Residuals
72.9	$88 - 72.9 = 15.1$
	4.3
	-13.7
	1.4
	5.4
	-12.7

Initial Residuals
16.8
4.8
-15.2
1.8
5.8
-14.2

RESIDUALS
HAVE GONE
DOWN!

XGBOOST: GRADIENT BOOSTING ALGORITHM

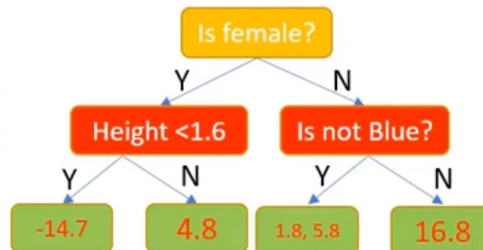


71.2

AVERAGE WEIGHT
(INITIAL GUESS FROM
PREVIOUS STEP)



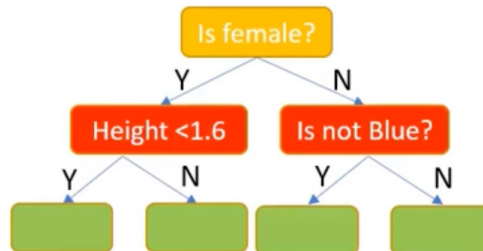
LEARNING RATE
0.1



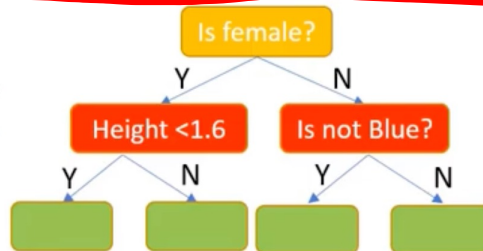
NOW YOU CAN
MAKE NEW
PREDICTIONS BY
COMBINING ALL
THE SCALED
PREDICTIONS
FROM ALL TREES



LEARNING RATE
0.1



LEARNING RATE
0.1



XGBOOST: WHAT IS THE EXTREME GRADIENT BOOSTING THEN? GREAT RESOURCES

Paper: <https://arxiv.org/pdf/1603.02754.pdf>
<https://xgboost.readthedocs.io/en/latest/tutorials/model.html>

XGBoost: A Scalable Tree Boosting System

Tianqi Chen
University of Washington
tqchen@cs.washington.edu

Carlos Guestrin
University of Washington
guestrin@cs.washington.edu

[cs.LG] 10 Jun 2016

ABSTRACT

Tree boosting is a highly effective and widely used machine learning method. In this paper, we describe a scalable end-to-end tree boosting system called XGBoost, which is used widely by data scientists to achieve state-of-the-art results on many machine learning challenges. We propose a novel sparsity-aware algorithm for sparse data and weighted quantile sketch for approximate tree learning. More importantly, we provide insights on cache access patterns, data compression and sharding to build a scalable tree boosting system. By combining these insights, XGBoost scales beyond billions of examples using far fewer resources than existing systems.

Keywords

Large-scale Machine Learning

problems. Besides being used as a stand-alone predictor, it is also incorporated into real-world production pipelines for ad click through rate prediction [15]. Finally, it is the de-facto choice of ensemble method and is used in challenges such as the Netflix prize [3].

In this paper, we describe XGBoost, a scalable machine learning system for tree boosting. The system is available as an open source package². The impact of the system has been widely recognized in a number of machine learning and data mining challenges. Take the challenges hosted by the machine learning competition site Kaggle for example. Among the 29 challenge winning solutions³ published at Kaggle's blog during 2015, 17 solutions used XGBoost. Among these solutions, eight solely used XGBoost to train the model, while most others combined XGBoost with neural nets in ensembles. For comparison, the second most popular method, deep neural nets, was used in 11 solutions. The success

XGBoost: A Scalable Tree Boosting System

Tianqi Chen
University of Washington
tqchen@cs.washington.edu

Carlos Guestrin
University of Washington
guestrin@cs.washington.edu

ABSTRACT

Tree boosting is a highly effective and widely used machine learning method. In this paper, we describe a scalable end-to-end tree boosting system called XGBoost, which is used widely by data scientists to achieve state-of-the-art results on many machine learning challenges. We propose a novel sparsity-aware algorithm for sparse data and weighted quantile sketch for approximate tree learning. More importantly, we provide insights on cache access patterns, data compression and sharding to build a scalable tree boosting system. By combining these insights, XGBoost scales beyond billions of examples using far fewer resources than existing systems.

Keywords

Large-scale Machine Learning

problems. Besides being used as a stand-alone predictor, it is also incorporated into real-world production pipelines for ad click through rate prediction [15]. Finally, it is the de facto choice of ensemble method and is used in challenges such as the Netflix prize [3].

In this paper, we describe XGBoost, a scalable machine learning system for tree boosting. The system is available as an open source package². The impact of the system has been widely recognized in a number of machine learning and data mining challenges. Take the challenges hosted by the machine learning competition site Kaggle for example. Among the 29 challenge winning solutions³ published at Kaggle's blog during 2015, 17 solutions used XGBoost. Among these solutions, eight solely used XGBoost to train the model while most others combined XGBoost with neural nets in ensembles. For comparison, the second most popular method deep neural nets, was used in 11 solutions. The success

XGBOOST: PAPER HIGHLIGHTS



- “The most important factor behind the success of XGBoost is its scalability in all scenarios. The system runs more than **ten times faster than existing popular solutions** on a single machine and **scales to billions of examples** in **distributed or memory-limited** settings.”
- “The scalability of XGBoost is due to several important systems and **algorithmic optimizations**. These innovations include: **a novel tree learning algorithm** is for handling **sparse data**; a theoretically justified **weighted quantile sketch procedure** enables handling instance weights in approximate tree learning. **Parallel and distributed computing** makes learning faster which enables quicker model exploration”.
- “More importantly, XGBoost exploits **out-of-core computation** and enables data scientists to process hundred millions of examples on a desktop”.
- “Finally, it is even more exciting to combine these techniques to make an end-to-end system that scales to even larger data with the least amount of cluster resources”.

Paper: <https://arxiv.org/pdf/1603.02754.pdf>



Thank You!